ASSIGNMENT 1

DESIGN, IMPLEMENT AND VALIDATE THE FARE LOGIC OF A CITY METRO SMART-CARD SYSTEM

Student Details	
Name	Nirmal Kumar T
Register Number	1972260020

Programme	B.Tech – Computer Science and Engineering
Course	Object Oriented Programming with C++
Assignment	CO1 – 50 Marks

Scenario: Namma Metro Smart-Card Fare System

Consolidated report covering Tasks 1–12, including analysis, C++ implementation, validation, complexity analysis and the Day-4 design challenge.

Submitted as an individual assignment

Assignment Overview

This assignment evaluates the design of a metro smart-card fare system using C++ fundamentals and object-oriented programming. The network has Inner, Middle and Outer zones with base fares of Rs 10, Rs 20 and Rs 30. Peak periods are 8:00–10:00 AM and 5:00–8:00 PM, with a 20% surcharge. A journey is permitted only when at least Rs 50 remains after fare deduction. Students and senior citizens receive a concession; because the original brief does not specify the concession percentage, this report states an explicit 10% illustrative assumption that can be replaced by the faculty-prescribed value.

Money representation: all monetary calculations use integer paise to avoid binary floating-point rounding. The program uses 100 paise = Rs 1.

Assumptions: the fare is based on the farthest zone reached; the peak surcharge is applied before the concession; concession is 10% for demonstration; weekend discount and monthly-pass rules are introduced only in Task 12. If the department specifies a different concession rate or order of application, update the named constants and expected test results accordingly.

Task 1 – Understand the Problem and Identify the Objects (4 Marks)

The system can be modelled using the following objects:

Object	Data	Behaviour / responsibility
SmartCard	Card ID, balance in paise, pass details	Check eligibility, deduct fare, recharge without allowing invalid balances
Passenger	Passenger ID, passenger category	Identify student/senior/regular concession eligibility
Journey	Origin, destination zone, time, date	Represent one trip and provide journey details
Zone	Zone category and associated fare	Represent Inner, Middle or Outer travel zone
FareTable	Base fares, surcharge and concession rates	Return the correct fare rules for a journey

In a procedural design, data is stored in separate variables and free functions operate on that data. In an object-oriented design, related data and behaviour are grouped into classes. This reduces duplicated logic and makes changes easier to isolate.

- **Encapsulation:** SmartCard keeps balance private and exposes controlled methods such as canTravel() and deduct().
- **Abstraction:** a gate asks the card whether travel is permitted without needing to know the internal balance-validation steps.
- **Inheritance:** StudentPassenger and SeniorPassenger may derive from Passenger and provide their concession rules.
- **Polymorphism:** a virtual concessionRate() function can return a category-specific rate through a Passenger reference.

Journey invariants: a zone and hour must be valid; fare must not be negative; a journey is accepted only if balance – fare is at least Rs 50; rejected journeys do not change the balance; money remains in integer paise.

Alternative – procedural style: use a balance variable, fare constants and functions such as calculateFare(), canTravel() and deductFare(). Both procedural and OOP implementations take O(1) time per journey and O(1) extra space per card. A rule change may require edits to several free functions, while a class-based design centralizes the rule inside FareTable or SmartCard.

Task 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 Marks)

The five token categories are keywords (class, constexpr, enum, return), identifiers (SmartCard, baseFarePaise, calculateFare), constants/literals (1000, 20, true), operators (+, -, /, >=, &&) and punctuators (;, {}, (), []).

#define performs preprocessor text substitution and does not provide ordinary type checking or C++ scope. **const** declares a non-modifiable typed object. **constexpr** identifies a value that can be evaluated at compile time when its initializer permits. constexpr is preferred for fixed fares and limits.

enum is unscoped and its enumerators can enter the surrounding scope. **enum class** is scoped and does not implicitly convert to an integer, so Zone::Inner is safer than an unscoped Inner.

Identifiers cannot begin with a digit, cannot be keywords and are case-sensitive. Use camelCase for variables/functions, PascalCase for types and uppercase snake case for constants.

Alternative and complexity: macros occupy no object storage, while constexpr values usually need no separate storage when used as constant expressions (though an object may require storage if odr-used). Both approaches have O(1) fare

lookup and do not materially change runtime. constexpr and enum class offer stronger type checking and scope safety.

Task 3 – Choose Data Types and Check Type Compatibility (4 Marks)

Value	Suggested type	Reason
Fare / balance	long long (paise)	Exact integer arithmetic; avoids decimal rounding
Peak flag	bool	Only true or false
Zone	enum class Zone	Restricted set of named values
Hour	int	0 through 23
Journey count	long long	Supports large network totals
Passenger category	enum class	Small fixed set of categories
Card identifier	std::string	Text identifier

The sizeof values are implementation-dependent; measure them on the target compiler using sizeof(type). Typical desktop g++ systems report int = 4 bytes, long long = 8 bytes, double = 8 bytes and bool = 1 byte. These are common results, not universal guarantees.

```
#include <iostream>
#include <climits>
int main() {
    std::cout << "sizeof(int) = " << sizeof(int) << '\n';
    std::cout << "sizeof(long long) = " << sizeof(long long) << '\n';
    std::cout << "sizeof(double) = " << sizeof(double) << '\n';
    std::cout << "INT_MAX = " << INT_MAX << '\n';
    std::cout << "LLONG_MAX = " << LLONG_MAX << '\n';
}
```

Integer arithmetic converts operands according to the usual arithmetic conversions. For example, combining int and double generally converts the int to double. A bool used in arithmetic is promoted to an integer value of 0 or 1.

At 3,000,000 journeys with a maximum base fare of Rs 30, the base-fare collection is Rs 90,000,000 (90 million rupees), or 9,000,000,000 paise. This exceeds the usual 32-bit signed int maximum of 2,147,483,647 paise, so long long is required for a paise total.

Alternative – double rupees: both double addition and long long addition are O(1), and both commonly use 8 bytes. However, binary floating point cannot represent many decimal fractions exactly. Integer paise is exact for whole-paise amounts. A rounding experiment can compare a repeated decimal addition with integer paise; record the observed compiler output rather than assuming a particular accumulated error.

Task 4 – Control Implicit Conversions (4 Marks)

Implicit conversions can occur in integer division, mixed arithmetic, conditions and signed/unsigned comparisons. Use explicit casts when a conversion is intentional. Brace initialization rejects narrowing conversions such as int x{3.7};.

Predict the output before execution

```
int a = 7, b = 2;
double c = a / b;                    // 3.0: integer division first
double d = static_cast<double>(a) / b; // 3.5
char ch = 'A' + 2;                   // 'C'
bool flag = 5;                       // true
unsigned int u = 1;
int s = -1;
std::cout << c << ' ' << d << '\n';
std::cout << ch << ' ' << ch + 1 << '\n';
std::cout << flag << ' ' << std::boolalpha << flag << '\n';
std::cout << (s < u ? "s < u" : "s >= u") << '\n';
```

Predicted output:

```
3 3.5
C 68
1 true
s >= u
```

Explanation: a / b is integer division, so it yields 3 before conversion to double. The cast makes the second division floating point. ch + 1 promotes char to int, so 'C' is printed followed by its character code 67 + 1 = 68. A non-zero integer converts to true. In the signed/unsigned comparison, s is converted to unsigned; its value becomes a large unsigned number, so s < u is false on the usual unsigned conversion rules.

Alternative: explicitly cast values and use brace initialization. This does not change O(1) time or storage in ordinary cases, but catches narrowing mistakes at compile time.

Task 5 – Write Correct Expressions with the Right Precedence (4 Marks)

Peak-hour rule: `(hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)`. The upper endpoints are excluded: 10:00 AM and 8:00 PM are off-peak.

Surcharge: `farePaise += farePaise * PEAK_SURCHARGE_PERCENT / 100;`. Concession can be applied with integer arithmetic, for example `farePaise = farePaise * (100 - concessionPercent) / 100;`.

Multiplication and division bind more tightly than addition; && binds more tightly than ||. Parentheses improve readability. Post-increment returns the old value, pre-increment returns the incremented value. Avoid expressions such as i = i++ because they are error-prone and may have undefined behaviour depending on the language version and expression form. Integer division truncates toward zero; the remainder has the sign of the dividend.

Predicted output

```
int x = 5, y = 3;
std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';
int z = x++ + ++y;
std::cout << z << ' ' << x << ' ' << y << '\n';
std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
std::cout << 10 / 3 << ' ' << 10 % 3 << ' '
          << -10 / 3 << ' ' << -10 % 3 << '\n';
int k = 0;
for (int i = 0; i < 10; i += 3) k += i;
std::cout << k << '\n';
```

Predicted output:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Alternative: a bool peak[24] lookup table still gives O(1) checks, with 24 boolean entries of storage. It can reduce repeated comparisons but uses extra space and needs careful initialization.

Task 6 – Design the Control Structures (4 Marks)

Use switch to select a base fare from the zone, if/else to apply surcharge and eligibility decisions, and loops to process journeys. A for loop is suitable when the number of journeys is known; a range-based for loop is convenient for an array or container. A while loop is useful when processing continues until input ends. Use continue to skip invalid input and break when a fatal condition occurs.

Pseudocode

```
BEGIN
  READ zone, hour, passengerCategory, balance
  IF zone is invalid OR hour is outside 0..23 THEN
      PRINT invalid journey
      STOP
  END IF
  SELECT base fare USING zone
  peak <- (hour >= 8 AND hour < 10) OR
          (hour >= 17 AND hour < 20)
  IF peak THEN add 20 percent surcharge
  IF passenger is eligible THEN apply concession
  IF balance - fare >= minimum balance THEN
      deduct fare and print accepted journey
  ELSE
      print denied journey
  END IF
END
```

The following switch-based helper is used in the complete implementation in Task 9. The default branch rejects an unknown zone rather than silently selecting a fare.

Task 7 – From Procedural to Object-Oriented (4 Marks)

A procedural version stores balance in a variable and passes it to free functions. An OOP version keeps balance private inside SmartCard and exposes canTravel() and deduct(). Private state prevents unrelated code from directly setting a negative balance.

```
class SmartCard {
private:
    long long balancePaise;
```

```
    static constexpr long long MIN_BALANCE_PAISE = 5000;
public:
    explicit SmartCard(long long initial) : balancePaise(initial) {}
    long long balance() const { return balancePaise; }
    bool canTravel(long long fare) const {
        return fare >= 0 && balancePaise - fare >= MIN_BALANCE_PAISE;
    }
    bool deduct(long long fare) {
        if (!canTravel(fare)) return false;
        balancePaise -= fare;
        return true;
    }
    bool recharge(long long amount) {
        if (amount <= 0) return false;
        balancePaise += amount;
        return true;
    }
};
```

Both versions have O(1) time for a single balance check/deduction and O(1) space per card. Private data does not make the arithmetic faster; it protects the invariant. If balance were public, any part of the program could bypass validation and break the minimum-balance rule.

Task 8 – Read and Print Data Safely (4 Marks)

Read input with cin and check the stream state. If extraction fails, call cin.clear() to reset the error flags and discard the invalid line with cin.ignore(numeric_limits::max(), '\\n'). Use iomanip manipulators such as setw and setfill for paise display. fixed and setprecision are useful when displaying double values, but money calculations should remain integer-based.

```
#include <iostream>
#include <limits>
#include <iomanip>

int main() {
    int hour;
    std::cout << "Enter hour (0-23): ";
    if (!(std::cin >> hour)) {
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Invalid numeric input\n";
        return 1;
    }
    if (hour < 0 || hour > 23) {
        std::cout << "Hour out of range\n";
        return 1;
    }
    long long paise = 3600;
    std::cout << "Fare Rs " << paise / 100 << '.'
              << std::setw(2) << std::setfill('0')
              << paise % 100 << '\n';
}
```

Alternative: getline() reads a complete line and stoi() converts it inside try/catch. Both approaches take O(L) time for a line of length L; getline requires a buffer proportional to L. cin extraction is convenient for simple numeric input, while getline plus stoi gives control over full-line validation.

Task 9 – Implement and Test the Fare Logic (4 Marks)

The following complete C++17 program implements the core three-zone fare rules, integer paise, a 20% peak surcharge, a 10% illustrative concession, safe card balance rules, validation and journey processing.

```
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>

enum class Zone { Inner, Middle, Outer };
enum class PassengerType { Regular, Student, Senior };

constexpr long long BASE_FARE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int CONCESSION_PERCENT = 10; // stated assumption
constexpr long long MIN_BALANCE_PAISE = 5000;

bool isPeakHour(int hour) {
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}

long long baseFare(Zone zone) {
    switch (zone) {
        case Zone::Inner: return BASE_FARE_PAISE[0];
        case Zone::Middle: return BASE_FARE_PAISE[1];
        case Zone::Outer: return BASE_FARE_PAISE[2];
    }
```

```
    return -1; // invalid zone
}

long long calculateFare(Zone zone, int hour, PassengerType type) {
    if (hour < 0 || hour > 23) return -1;
    long long fare = baseFare(zone);
    if (fare < 0) return -1;

    if (isPeakHour(hour))
        fare += fare * PEAK_SURCHARGE_PERCENT / 100;

    if (type == PassengerType::Student ||
        type == PassengerType::Senior)
        fare = fare * (100 - CONCESSION_PERCENT) / 100;

    return fare;
}

class SmartCard {
private:
    long long balancePaise;
public:
    explicit SmartCard(long long initial) : balancePaise(initial) {}
    long long balance() const { return balancePaise; }
    bool canTravel(long long fare) const {
        return fare >= 0 &&
               balancePaise - fare >= MIN_BALANCE_PAISE;
    }
    bool deduct(long long fare) {
        if (!canTravel(fare)) return false;
        balancePaise -= fare;
        return true;
    }
};

void printMoney(long long paise) {
    std::cout << "Rs " << paise / 100 << '.'
              << std::setw(2) << std::setfill('0')
              << paise % 100 << std::setfill(' ');
}

int main() {
    SmartCard card(10000); // Rs 100.00
    struct Trip { Zone zone; int hour; PassengerType type; };
    const std::vector<Trip> trips = {
        {Zone::Inner, 11, PassengerType::Regular},
        {Zone::Outer, 18, PassengerType::Regular},
        {Zone::Middle, 14, PassengerType::Regular},
        {Zone::Inner, 10, PassengerType::Student}
    };

    for (const auto& trip : trips) {
        long long fare = calculateFare(trip.zone, trip.hour, trip.type);
        if (fare < 0) {
            std::cout << "Invalid journey\n";
            continue;
        }
        if (card.deduct(fare)) {
            std::cout << "Accepted: ";
            printMoney(fare);
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << " | Balance: ";
        printMoney(card.balance());
        std::cout << '\n';
    }
}
```

Illustrative test table (expected values from the stated rules):

Test	Input / condition	Expected result
T1	Inner, 11:00, regular	Rs 10.00; off-peak
T2	Outer, 18:00, regular	Rs 36.00; peak
T3	Middle, 14:00, regular	Rs 20.00; off-peak
T4	Inner, 10:00, student	Rs 9.00; 10:00 is off-peak; 10% concession
T5	Hour = 8	Peak surcharge applies
T6	Hour = 10	Peak surcharge does not apply
T7	Hour = 17	Peak surcharge applies
T8	Hour = 20	Peak surcharge does not apply

Test	Input / condition	Expected result
T9	Balance after fare exactly Rs 50	Journey is accepted
T10	Unknown zone / hour outside 0–23	Rejected as invalid
T11	Negative recharge	Rejected; balance unchanged

For the five trips in main(), the first fares are Rs 10.00, Rs 36.00 and Rs 20.00. Starting from Rs 100.00, the balance becomes Rs 90.00, then Rs 54.00. The third journey is denied because paying Rs 20.00 would leave Rs 34.00, below the required Rs 50.00. The fourth trip is then also denied because the balance remains Rs 54.00 and paying Rs 9.00 would leave Rs 45.00.

Execution evidence: compile and run this source on your system. Capture the compiler command, compiler version and actual output screenshot. The table above is an expected-results analysis, not a claim that your local execution has already been performed.

Task 10 – Compare Expectations with Execution (4 Marks)

Compare integer-paise totals with double-rupee totals for 20,000, 500,000 and 3,000,000 journeys. For a constant Rs 30 fare, the corresponding totals are Rs 600,000, Rs 15,000,000 and Rs 90,000,000. In paise these are 60,000,000; 1,500,000,000; and 9,000,000,000. The final paise total exceeds a typical 32-bit signed int limit, so int is unsafe.

Journeys	Total at Rs 30 each	Total in paise	32-bit int paise safe?
20,000	Rs 600,000	60,000,000	Yes
500,000	Rs 15,000,000	1,500,000,000	Yes, within typical INT_MAX
3,000,000	Rs 90,000,000	9,000,000,000	No

A long long is appropriate for large paise totals. A double has a wide range but may represent decimal rupee fractions approximately. With -Wall -Wextra, warnings may identify signed/unsigned comparisons, unused variables, missing return paths or suspicious conversions. The exact warnings depend on the source and compiler; report only warnings actually observed.

Alternative: int may overflow for the network paise total; long long safely holds the shown total; double avoids integer overflow at this scale but can introduce rounding. Addition is O(1) per journey for all three types, and total accumulation is O(n) for n journeys.

Task 11 – Recommend and Defend the Final Design (4 Marks)

The recommended design uses integer paise stored in long long, constexpr constants for fixed fares and limits, enum class for zones and passenger types, a FareTable/fare calculation component, and a SmartCard class with private balance. A separate Passenger hierarchy or category policy can supply concession rules if more passenger categories are added.

A fare revision should be made by changing the central fare table and named rule constants, then running the boundary and regression tests. The design is suitable for millions of journeys because each fare calculation and card eligibility check is O(1).

Limitations include the assumed concession percentage, a fixed in-program fare table, simplified passenger identity verification and no persistence/concurrency layer. A production system should load validated configuration, audit transactions, protect card IDs and handle simultaneous gate requests.

Alternative – configuration file: loading z zone fares at startup costs O(z) time and O(z) storage. It avoids recompilation for fare changes but requires validation, secure file handling and a defined fallback if the file is missing or invalid.

Task 12 – Re-evaluate and Adapt the Design: Day-4 Challenge (8 Marks)

Changed requirements: add Airport zone at Rs 60, a 10% weekend discount that does not combine with peak surcharge, and a monthly pass allowing unlimited journeys in two chosen zones.

A. Impact analysis (2 Marks)

• Add Zone::Airport and a fourth fare entry of 6000 paise.

• Update zone validation and every switch or fare lookup that assumes exactly three zones.

• Add a weekend rule and explicitly ensure weekend discount and peak surcharge are mutually exclusive.

• Add a Pass class containing two permitted zones and an active/expiry state.

B. Evaluate the original design (2 Marks)

The original switch can be extended with another case, but every switch must be found and updated. A table indexed by zone makes fare extension easier, provided the enum values map safely to valid indices. SmartCard remains useful for stored balance, but monthly-pass eligibility is a separate responsibility and should not be mixed into balance deduction.

Worked examples: Airport off-peak weekday fare = Rs 60.00. Airport peak weekday fare = Rs 72.00. Airport weekend fare = Rs 54.00 if the weekend discount applies instead of, and not in addition to, the peak surcharge. A valid monthly pass for Airport makes an Airport journey fare-free while the pass is active.

C. Adapted design and implementation (2 Marks)

```
#include <array>
#include <iostream>
#include <optional>

enum class Zone { Inner, Middle, Outer, Airport };

constexpr std::array<long long, 4> FARES = {
    1000, 2000, 3000, 6000
};
constexpr int PEAK_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;

class MonthlyPass {
private:
    Zone firstZone;
    Zone secondZone;
    bool active;
public:
    MonthlyPass(Zone a, Zone b, bool isActive)
        : firstZone(a), secondZone(b), active(isActive) {}
    bool covers(Zone z) const {
        return active && (z == firstZone || z == secondZone);
    }
};

long long revisedFare(Zone zone, bool peak, bool weekend,
                      const MonthlyPass* pass = nullptr) {
    if (pass != nullptr && pass->covers(zone)) return 0;

    const auto index = static_cast<std::size_t>(zone);
    if (index >= FARES.size()) return -1;

    long long fare = FARES[index];
    if (weekend) {
        // Weekend discount replaces the peak surcharge.
        fare = fare * (100 - WEEKEND_DISCOUNT_PERCENT) / 100;
    } else if (peak) {
        fare += fare * PEAK_PERCENT / 100;
    }
    return fare;
}

int main() {
    MonthlyPass pass(Zone::Inner, Zone::Airport, true);
    std::cout << "Airport weekday peak: "
              << revisedFare(Zone::Airport, true, false) << " paise\n";
    std::cout << "Airport weekend: "
              << revisedFare(Zone::Airport, true, true) << " paise\n";
    std::cout << "Airport with pass: "
              << revisedFare(Zone::Airport, false, false, &pass) << " paise\n";
}
```

Expected output:

```
Airport weekday peak: 7200 paise
Airport weekend: 5400 paise
Airport with pass: 0 paise
```

The enum declaration order matches the fare array indices. In a production implementation, use an explicit conversion/lookup function rather than relying on enum ordinal values if the enum may be reordered. The pass also needs start date, expiry date, passenger/card association and validity checks; these are represented by a simple active flag here to demonstrate the concept.

D. Evidence-based defence (2 Marks)

The recommended decision is MODIFY, not fully redesign. Retain SmartCard for balance protection and add MonthlyPass as a separate class. Replace repeated switch logic with a centralized fare table and separate rule application. This reduces the number of code locations changed when a zone is added. Validation must include all four zones, peak and off-peak periods, weekends, pass-covered and uncovered zones, expired passes and invalid zone values.

Validation	Expected fare / behaviour
Airport weekday off-peak	6000 paise (Rs 60.00)
Airport weekday peak	7200 paise (Rs 72.00)
Airport weekend during peak time	5400 paise (Rs 54.00); weekend rule wins
Airport covered by active pass	0 paise
Zone not in fare table	Invalid fare; journey rejected

Overall Conclusion

The assignment demonstrates how C++ fundamentals support a reliable metro fare application. Integer paise prevents monetary rounding, constexpr centralizes fixed rules, enum class models zones safely, control structures implement the fare conditions, and SmartCard encapsulation protects the minimum-balance invariant. The extended design accommodates an Airport zone, weekend pricing and monthly passes by modifying the fare policy and adding a dedicated pass class. The final approach is maintainable and has constant-time fare calculation for each journey.

Submission Checklist

• Create repository named DSA01-1972260020-Assignment1.

• Keep one folder for each task (Task01 through Task12) with source files, inputs and output screenshots.

• Include README.md with student details, compiler/version, exact compile command, run instructions and a short result note for every task.

• Commit progressively, with at least one commit per task.

• Add the working GitHub repository link to the report cover page and submit it to the faculty.

• Run every program yourself and replace illustrative expected output with your actual execution evidence where required.